

Score Breakdown (10-Point Scale)

Section I (Multiple-Choice): 20 items $\times$ 0.25 pts = **5.0 pts**

Section II (Short-Answer): 4 questions = **5.0 pts**

Total Exam Score: 10.0 points

**NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE & AI**

OFFICIAL SOLUTIONS & GRADING GUIDE

Course: Data Analysis with Python (DSAI1005)

Content: Midterm Exam (Weeks 1 – 4) — **Exam Code:**
201-EN

SECTION I: MULTIPLE-CHOICE KEY (5.0 pts — 0.25 pts each)

Q	1	2	3	4	5	6	7	8	9	10
Key	B	C	A	B	B	B	A	B	B	B

Q	11	12	13	14	15	16	17	18	19	20
Key	B	B	B	C	B	B	A	B	B	B

Concise Conceptual Explanations:

- **Q1 (B):** EDA (pioneered by John Tukey) seeks to explore data structure, test assumptions, discover patterns, and spot anomalies before modeling.
- **Q2 (C):** Scale property of standard deviation: multiplying all values by a constant $c = 3$ scales the standard deviation by $|c| = 3$ ($\text{Std}(cX) = |c|\text{Std}(X)$).
- **Q3 (A):** Tukey's standard box plot defines outer fences as $[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}]$.
- **Q4 (B):** $r = 0$ indicates the absence of linear correlation; it does not preclude quadratic or other deterministic non-linear dependencies.
- **Q5 (B):** `np.arange(start=3, stop=15, step=4)` generates $[3, 7, 11]$ (stop value 15 is excluded).
- **Q6 (B):** `m[::-1, 1]` reverses the row order: $[7, 8, 9] \rightarrow [4, 5, 6] \rightarrow [1, 2, 3]$. Extracting column index 1 yields $[8, 5, 2]$.
- **Q7 (A):** Broadcasting rules: Dim 1: $5 \text{ vs } 1 \rightarrow 5$; Dim 2: $1 \text{ vs } 4 \rightarrow 4$; Dim 3: $3 \text{ vs } 1$ (implicit trailing) $\rightarrow 3 \Rightarrow \text{shape } (5, 4, 3)$.
- **Q8 (B):** `np.where(condition, x, y)` is the vectorized equivalent of the ternary operator.
- **Q9 (B):** `axis=0` computes the sum down the rows (collapsing rows, preserving columns), producing a 1D array of shape $(5,)$.
- **Q10 (B):** A `pd.Series` is a 1D labeled array capable of holding any data type.
- **Q11 (B):** Label-based slicing with `.loc` is inclusive of both the start and stop boundaries $\rightarrow$ includes 'a', 'b', and 'c'.
- **Q12 (B):** `df.fillna()` replaces missing entries (NaN) with designated constant values or summary statistics.
- **Q13 (B):** In Pandas, multiple boolean conditions require bitwise operators (`|`) and explicit parentheses around each clause.
- **Q14 (C):** The 'IT' group has salaries 50 and 70; the maximum value is 70.
- **Q15 (B):** An inner join retains only those rows where the join key is mutually present in both DataFrames.

- **Q16 (B):** `plt.subplots(2, 3)` creates a 2×3 grid of 6 individual Axes subplots.
- **Q17 (A):** `sns.kdeplot()` estimates and visualizes the smooth probability density function of continuous variables.
- **Q18 (B):** `sns.pairplot()` renders pairwise scatter plots across numeric columns with histograms/KDEs along the diagonal.
- **Q19 (B):** `plt.tight_layout()` automatically adjusts subplot padding to prevent overlapping titles, labels, and ticks.
- **Q20 (B):** `sns.barplot()` plots group means with bootstrapped confidence intervals (error bars) by default.

SECTION II: SOLUTIONS & GRADING RUBRIC (5.0 pts)

General Grading Policy for Evaluators

- Do not penalize students for interchange of single quotes ' ' and double quotes " ".
- Minor trailing syntax slips (e.g., missing closing parenthesis at the line end) should not incur deductions if the algorithmic logic is sound.
- Mathematically and logically equivalent implementations in NumPy / Pandas / Matplotlib are entitled to full credit.

Question 1: NumPy Vectorization & Slicing (1.0 pt)

a) (0.5 pts):

- **Mechanism:** Array ****Broadcasting****. (0.25 pts)
- **Resulting Matrix:** (0.25 pts)

$$\text{weighted_scores} = \begin{bmatrix} 8.0 \times 0.2 & 7.0 \times 0.3 & 9.0 \times 0.5 \\ 6.0 \times 0.2 & 8.0 \times 0.3 & 7.0 \times 0.5 \\ 9.0 \times 0.2 & 9.0 \times 0.3 & 8.5 \times 0.5 \\ 5.0 \times 0.2 & 6.0 \times 0.3 & 6.5 \times 0.5 \end{bmatrix} = \begin{bmatrix} 1.6 & 2.1 & 4.5 \\ 1.2 & 2.4 & 3.5 \\ 1.8 & 2.7 & 4.25 \\ 1.0 & 1.8 & 3.25 \end{bmatrix}$$

b) (0.5 pts):

- **NumPy statement for gpa:** (0.25 pts)

```
gpa = np.sum(weighted_scores, axis=1) # Or: weighted_scores.sum(axis=1)
```

- Row-wise sums yield: `[8.2, 7.1, 8.75, 6.05]`.
- Condition `gpa >= 8.0` is true for indices 0 and 2.
- Extracting column 0 (scores[... , 0]): **Printed output:** `[8.0, 9.0]` (or `[8., 9.]`). (0.25 pts)

Question 2: Data Cleaning & Aggregation with Pandas (1.5 pts)

a) (0.5 pts): Filtering and missing value imputation:

```
df_filtered = df_trans[(df_trans['status'] == 'SUCCESS') & (df_trans['amount'] >= 5_000_000)].copy()
df_filtered['fee'] = df_filtered['fee'].fillna(0)
```

Rubric: Correct boolean filtering with bitwise `&`: 0.25 pts; Correct `.fillna(0)`: 0.25 pts.

b) (1.0 pts): Groupby and aggregation expression:

```
df_branch_summary = df_filtered.groupby('branch').agg(  
    total_amount=('amount', 'sum'),  
    avg_fee=('fee', 'mean'),  
    trans_count=('amount', 'count') # Or 'size'  
) .sort_values(by='total_amount', ascending=False)
```

Rubric: Correct `groupby('branch')`: 0.25 pts; Correct aggregations ('sum', 'mean', 'count'): 0.5 pts; Correct `sort_values(..., ascending=False)`: 0.25 pts.

Question 3: Multi-plot Layouts with Matplotlib (1.0 pt)

a) (0.5 pts): Statement to initialize subplots:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

Rubric: Correct grid specification (1, 2): 0.25 pts; Correct `figsize=(12, 4)`: 0.25 pts.

b) (0.5 pts): Plotting and layout commands:

```
axes[0].plot(df_kpi['month'], df_kpi['profit'], color='blue', marker='o')  
axes[1].bar(df_kpi['month'], df_kpi['profit'], color='steelblue')  
plt.tight_layout()
```

Rubric: Correct line plot on `axes[0]` with circular marker: 0.2 pts; Correct bar chart on `axes[1]`: 0.2 pts; Correct `plt.tight_layout()`: 0.1 pts.

Question 4: Exploratory Visualization & Business Insights (EDA) (1.5 pts)

a) (0.75 pts):

- **Chart type:** ****Box Plot**** (or Violin Plot). (0.25 pts)
- **Seaborn statement:** (0.5 pts)

```
sns.boxplot(data=df_cards, x='is_fraud', y='tx_amount', hue='card_type')  
# (Acceptable variant: x='card_type', y='tx_amount', hue='is_fraud')
```

b) (0.75 pts) — Analytical Takeaway & Policy Recommendation:

- *Analytical Takeaway (0.375 pts):* Credit card fraud typically targets high-credit limits with anomalous transaction sizes (extreme outliers between 50M and 200M VND), contrasting sharply with regular transactions (≤ 10 M VND) and debit card behavior.
- *Policy Recommendation (0.375 pts):* The bank should establish automated step-up authentication (Smart OTP/biometrics) or immediate transaction freezes for all single credit card charges exceeding 50 million VND.

————— **END OF SOLUTIONS** —————